

GraphPoint: Semantic Entity Graphs and Point Trajectories for Compositional Robot Manipulation

Kang Luo, Hesheng Wang[†]

Abstract—

I. INTRODUCTION

II. RELATED WORK

A. Visual and Point-Based Manipulation Policies

Visual imitation learning provides a practical route from demonstrations to robot control. ACT [1] predicts action chunks for fine-grained manipulation, while Diffusion Policy [2] models action sequences through conditional denoising. At larger scale, $\pi_{0.5}$ [3] combines heterogeneous training data with vision-language-action modeling to support generalization to new environments. These approaches establish strong image-conditioned policies, while 3D representations offer a complementary geometric interface. DP3 [4] conditions diffusion policies on compact features extracted from sparse scene point clouds. Point Policy [5] represents observations and actions using semantic key points, enabling policy learning from human demonstration videos. More recently, Point Bridge [6] uses unified point representations for synthetic-data training and sim-to-real transfer. GraphPoint builds on this point-based perspective, organizing task-relevant observations by semantic entity roles and predicting actor point trajectories. Our focus is how this organization supports instruction-dependent task composition.

B. Object-Centric and Graph-Based Representations

Object-centric representations expose individual entities as reusable components of a scene. GROOT [7] combines segmented object point clouds with a transformer policy and segmentation correspondence to handle changes in backgrounds, viewpoints, and object instances. Graph-based approaches further represent interactions between entities. GraphMimic [8] abstracts videos into object and visual-action vertices, pretrains a model to generate future graphs, and conditions a control policy on these predictions. Compose by Focus [9] selects task-relevant scene subgraphs, encodes them with a graph attention network, and combines diffusion-based atomic skills with a vision-language task planner. These works establish object structure and task-relevant relations as useful priors for control and skill composition. GraphPoint studies a specific combination of actor/patient/target roles, explicit action-type and modifier conditioning, and point-trajectory generation. Its distinction lies in this representation and policy interface; object-centric

inputs and graph-based skill composition are established ingredients.

C. Compositional Generalization in Manipulation

Compositional generalization concerns new combinations of familiar task factors as well as new sequences of learned behaviors. CLIPort [10] couples pretrained semantic features with spatially precise action prediction for language-conditioned manipulation. VIMA [11] introduces multimodal task prompts and a benchmark with 4 generalization levels, while CompoSuite [12] explicitly combines robots, objects, obstacles, and objectives to evaluate unseen task combinations. LIBERO [13] studies declarative and procedural knowledge transfer across manipulation tasks, and CALVIN [14] evaluates language-conditioned continuous control over multi-step task sequences. These benchmarks address complementary forms of transfer whose difficulty depends on the factors held out during training. CoMani provides a focused LIBERO-based evaluation of modifier combinations at a fixed action type, action-type combinations at a fixed modifier, and unseen sequences of atomic tasks. Its Seq-ID and Seq-OOD sets distinguish composition of familiar atomic tasks from sequences containing a held-out atomic combination, while matched initial scenes reduce opportunities to infer the task from visual context alone.

III. METHOD

A. Overview

GraphPoint represents manipulation as interactions among 3D point entities and predicts future gripper points in the same geometric space. As shown in Fig. 1, language analysis, visual localization, and point tracking construct a task-conditioned entity graph, which a shared policy maps to a point trajectory and a subtask progress estimate. For a multi-step instruction, the system instantiates this graph for each subtask and uses predicted progress to advance through the sequence. The learned policy receives entity geometry and structured language conditions; RGB images are used by the perception front end.

B. Task-to-Graph Construction

Semantic decomposition. A language model decomposes an instruction into ordered subtasks, each specified by an action type, a modifier, and entity roles. We distinguish the *actor* A (the gripper), the *patient* P (the manipulated object or part), and the *target* T (the reference entity). For example, “put the cheese to the right of the stove” assigns cheese to P , stove to T , *put* to the action type, and *right* to

Kang Luo and Hesheng Wang are with IRMV Lab, the Department of Automation, Shanghai Jiao Tong University.

[†]Corresponding author email: wanghesheng@sjtu.edu.cn

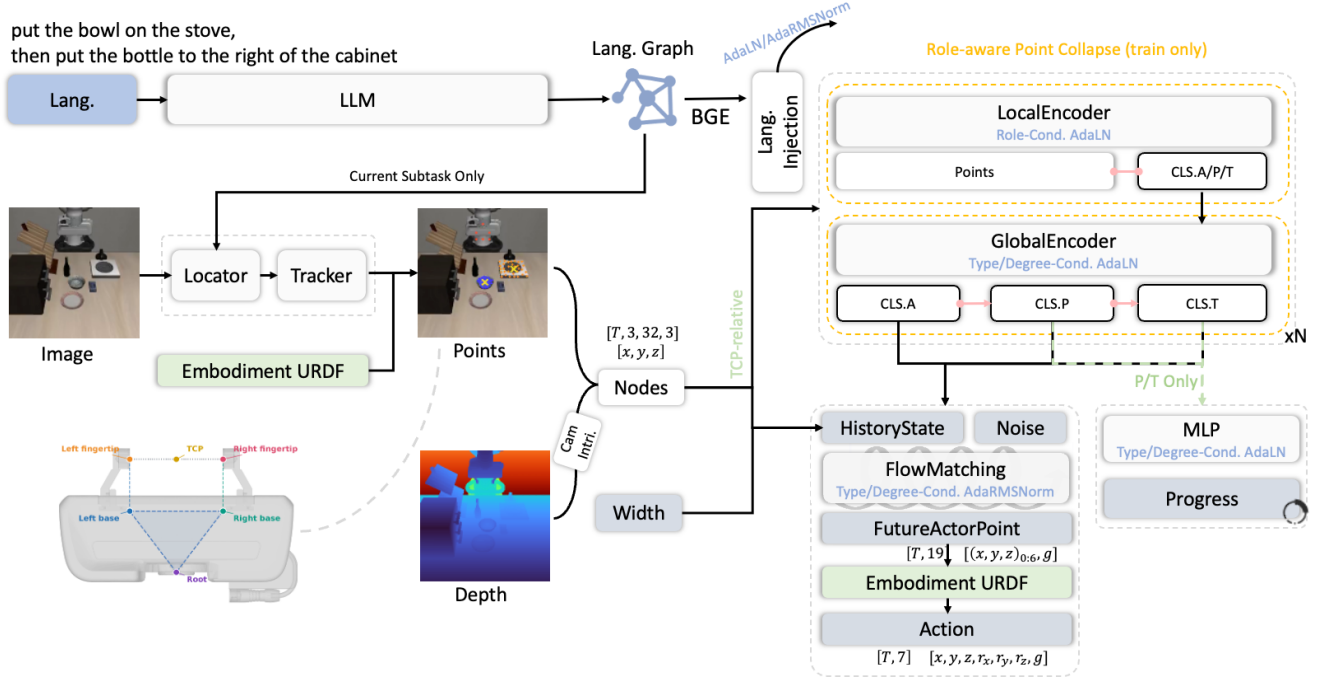


Fig. 1. GraphPoint framework. Language analysis identifies ordered subtasks and their semantic roles, while visual localization and tracking ground the relevant entities as 3D points. The policy alternates local entity encoding with temporal graph attention, conditioned on roles, action types, and modifiers. A flow decoder predicts future actor points and gripper commands, and a progress head supports subtask transitions. Robot geometry converts predicted points into executable gripper poses.

the modifier. Entity names guide visual grounding, whereas action types and modifiers are encoded separately using frozen BGE embeddings [15] and learned projections. We denote their concatenation by the task condition c and the learned embedding of role r by e_r . For readability, c also denotes the version with separately normalized action and modifier embeddings used by the prediction heads. Absent modifiers use a learned null embedding, and absent entities are masked.

Grounded point entities. A pretrained vision-language locator identifies the requested objects or parts, and SAM 2 [16] produces masks from the localization prompts. We sample 32 points per visual entity and track their image coordinates with TAPNext++ [17]. Depth and camera intrinsics back-project these tracks into 3D. The actor instead uses 6 keypoints computed from robot geometry and proprioception: the gripper root, TCP, 2 finger bases, and 2 fingertips. We align all entities in camera coordinates and subtract the current TCP position from every point in the observation history and prediction horizon. Thus, coordinates retain camera-axis orientation while sharing a translation origin at the current TCP. Actor points are padded with a validity mask, and coordinates are normalized using training-set quantiles. The policy observes 10 frames, including the current frame.

C. Role-Structured Entity Graph Encoder

Role-conditioned geometry and task-conditioned interaction. A shared point MLP embeds entity coordinates,

with keypoint identity embeddings added for the actor. Each entity at each observed time receives 4 learned CLS tokens to summarize its geometry. The encoder alternates local encoding within each entity and global interaction among entity summaries:

$$\begin{aligned} (\tilde{C}_r, F_r^{\ell+1}) &= \text{Local}_\ell(C_r^\ell, F_r^\ell; e_r), \\ C^{\ell+1} &= \text{Graph}_\ell(\{\tilde{C}_r\}_{r \in \{A, P, T\}}; c). \end{aligned} \quad (1)$$

Here, F_r and C_r denote point features and CLS summaries, respectively, with observation-time indices omitted for clarity. Local self-attention processes each entity and frame independently, conditioned on its role e_r . Graph attention exchanges only CLS summaries across entities and observed times, conditioned on the action and modifier c . Its connectivity follows the bidirectional chain $A \leftrightarrow P \leftrightarrow T$, with additional within-role temporal connections. Direct actor-target attention is masked, so their interaction is mediated by the patient through successive layers. Temporal rotary embeddings distinguish observation times. After 8 layers, the current-time CLS summaries form memory M_t , incorporating both geometry and interaction history.

Semantic injection. Both local and graph blocks contain attention and feed-forward sublayers, whose inputs are modulated through adaptive layer normalization:

$$\text{AdaLN}(h; q) = (1 + \gamma(q)) \odot \text{LN}(h) + \beta(q). \quad (2)$$

The scale γ and shift β are learned functions of the condition, broadcast over tokens. Using $q = e_r$ locally lets the encoder

interpret geometry according to whether it belongs to the actor, patient, or target. Using $q = c$ globally makes the same entity configuration task-dependent, such as distinguishing placing an object *on* a target from placing it to its *right*. Thus, role semantics control entity encoding, while action and modifier semantics control interaction through feature modulation.

Role-aware point collapse. During training, we independently collapse each patient or target point set with probability 0.05 to reduce dependence on detailed object shape. For the patient, all points are replaced by the mean of the 4 points nearest to the TCP; for the target, they are replaced by the entity centroid. This operation is applied consistently over the sampled temporal window, with anchors computed separately at each frame. It preserves point counts and validity masks and leaves actor points and future action supervision unchanged. The augmentation exposes the encoder to simplified geometry while retaining a role-specific spatial anchor.

D. Point-Trajectory Flow Policy

We predict $H = 10$ future steps, each containing 6 actor points and a scalar gripper command:

$$Y = \{(X_{t+h}^A, g_{t+h})\}_{h=1}^H. \quad (3)$$

Here, X_{t+h}^A contains actor coordinates relative to the current TCP, and g_{t+h} is the open/close command. Each step is represented by a 19-dimensional vector in point-wise XYZ order followed by the gripper command.

Geometry- and language-conditioned decoding. A 6-layer Transformer models this trajectory with conditional flow matching [18]. It embeds the observed actor history U_t together with the noisy future trajectory Z_τ and predicts a velocity field:

$$v_\theta = \text{Decoder}_\theta(Z_\tau, U_t; M_t, c, \tau). \quad (4)$$

Each decoder layer uses self-attention to relate trajectory steps, cross-attention to retrieve entity geometry from M_t , and a feed-forward sublayer to update features. Future tokens attend to both history and future, while history tokens cannot attend to future tokens. Temporal rotary embeddings encode their relative times. Action and modifier semantics enter every sublayer through AdaRMSNorm, following the scale-and-shift principle of Eq. (2) with RMS normalization. Separate mappings of the task condition c and flow-time embedding are added to generate the scale, shift, and residual gate. Consequently, geometry guides decoding through cross-attention, while language and flow time modulate how each layer updates the trajectory. A final projection of the future tokens produces the point and gripper velocity predictions.

Training and execution. For a demonstrated trajectory Y and Gaussian noise ϵ , the flow objective is

$$\begin{aligned} Z_\tau &= (1 - \tau)Y + \tau\epsilon, \\ \mathcal{L}_{\text{flow}} &= \mathbb{E}[\text{MSE}(v_\theta, \epsilon - Y)], \end{aligned} \quad (5)$$

where the squared error is averaged over trajectory steps and output dimensions. At inference, 10 Euler steps integrate

from noise at $\tau = 1$ to a trajectory at $\tau = 0$. After denormalization and restoring the TCP translation, we fit the robot’s gripper keypoint template to each predicted point set using least-squares rigid alignment. The template uses the observed gripper width, and the predicted gripper command is applied separately. The recovered poses are transformed into the control frame and executed in a receding-horizon manner. This geometric conversion provides the interface between point predictions and the robot controller.

E. Subtask Progress and Long-Horizon Composition

The progress head estimates how far the active subtask has advanced from the current patient and target summaries:

$$\hat{s}_t = \text{Progress}_\theta(C_{t,P}, C_{t,T}; c). \quad (6)$$

It uses a task-conditioned MLP with AdaLN and a sigmoid output, so the same geometry is evaluated under the active action and modifier. Training targets increase linearly from 0 to 1 within each annotated subtask interval, and the complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{flow}} + \text{SmoothL1}(\hat{s}_t, s_t). \quad (7)$$

At deployment, the planner advances when predicted progress exceeds a threshold for a consecutive window of policy calls. It performs a release-and-lift transition, selects the next subtask’s entities and language conditions, and resets policy history. The same policy is reused for every subtask, enabling execution of task sequences assembled from atomic demonstrations without requiring a separate policy for each sequence.

IV. CoMANI BENCHMARK

CoMani comprises 3 suites testing modifier, action-type, and sequential composition (Fig. 2; Table I). We generate 50 trajectories per atomic task using MimicGen [19]. Within each suite, initial scenes are matched wherever feasible to require instruction-dependent behavior and discourage fixed vision-to-action mappings. We report success rates by split over 30 trials per atomic task and 10 per sequence.

CoMani-Modifier. With pick-and-place (*put*) fixed, this suite combines 3 objects, 3 targets, and 2 modifiers (*on/right of*) into 18 tasks: 12 training/in-distribution (ID) and 6 out-of-distribution (OOD). Each OOD object–target pair appears in training with the opposite modifier, isolating generalization to unseen combinations of familiar factors. The ID/OOD comparison therefore tests whether a policy can apply the instructed relation to a familiar object–target pair beyond its demonstrated placement.

CoMani-Action. With *right* fixed, this suite varies *put*, *sweep*, and *rotate* across 18 tasks: 7 ID and 3 OOD. For *put/sweep*, each held-out object–target pair is demonstrated with the other action; for rotation, training uses the stove knob and OOD evaluation uses cream cheese. These tests examine whether a policy can reuse an action with familiar entities in a new combination and transfer rotation across different manipulated entities. Put/sweep pairs share initial

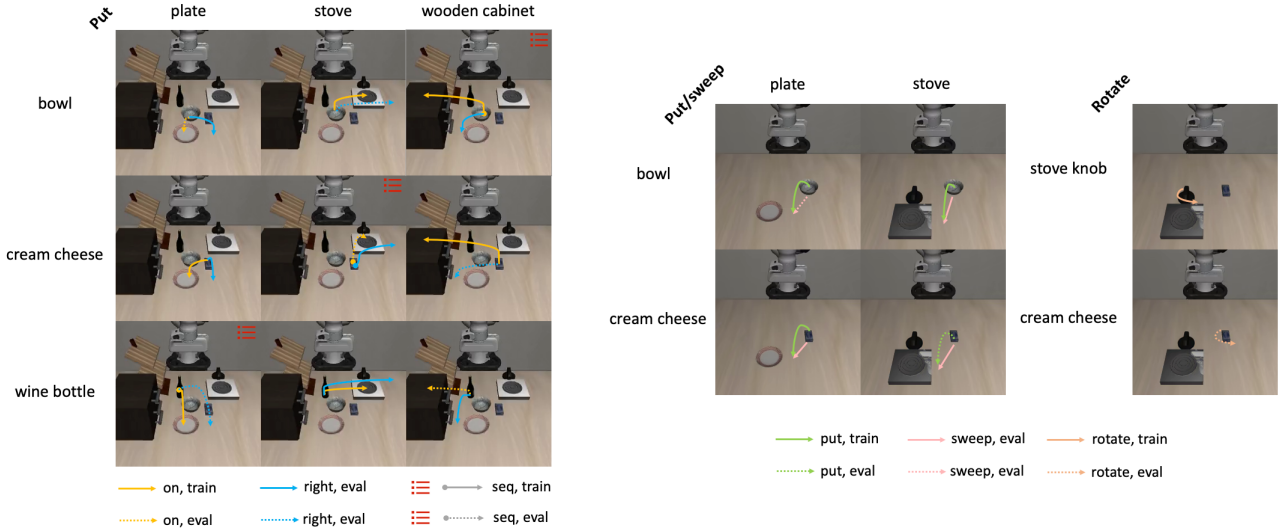


Fig. 2. CoMani task construction and splits. CoMani-Modifier (left) varies spatial relations with a fixed action type, while CoMani-Action (right) varies action types with a fixed modifier. Solid and dotted arrows indicate training and held-out combinations, respectively. In the left panel, filled circles at arrow starting points and red symbols at the upper-right corners of scenes highlight the atomic tasks selected for CoMani-Seq.

states and final-position goals, so success measures goal attainment rather than interaction mode.

CoMani-Seq. This suite tests whether policies learned from CoMani-Modifier atomic demonstrations can execute 3-step instructions without sequence-level demonstrations. *Seq-ID* combines 3 ID tasks; *Seq-OD* replaces the cream-cheese task with its OOD modifier variant, giving 2 ID tasks and 1 OOD task. Each set includes all 6 orderings, yielding 12 unseen sequences. Seq-ID tests ordered execution and subtask transitions among familiar skills; Seq-OD adds an unseen atomic combination while preserving the objects, targets, and sequence orderings. Comparing the 2 sets assesses how atomic compositional generalization carries over to long-horizon execution.

V. EXPERIMENTS

A. Experimental Setup

B. How Does GraphPoint Compare with Established Policies?

C. Does the Structured Entity Graph Enable Compositional Semantic Generalization?

D. Does Point-Based Action Representation Enable Cross-Embodiment Transfer?

E. Can Atomic Subtask Policies Compose into Long-Horizon Behaviors?

VI. CONCLUSION

REFERENCES

- [1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, 2023. [Online]. Available: <https://roboticsproceedings.org/rss19/p016.html>
- [2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion Policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems*, 2023. [Online]. Available: <https://diffusion-policy.cs.columbia.edu/>
- [3] Physical Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [4] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3D Diffusion Policy: Generalizable visuomotor policy learning via simple 3D representations,” in *Proceedings of Robotics: Science and Systems*, 2024. [Online]. Available: <https://3d-diffusion-policy.github.io/>
- [5] S. Haldar and L. Pinto, “Point Policy: Unifying observations and actions with key points for robot manipulation,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 1825–1846. [Online]. Available: <https://proceedings.mlr.press/v305/haldar25a.html>
- [6] S. Haldar, L. Johannsmeier, L. Pinto, A. Gupta, D. Fox, Y. Narang, and A. Mandlekar, “Point Bridge: 3D representations for cross domain policy learning,” *arXiv preprint arXiv:2601.16212*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.16212>
- [7] Y. Zhu, Z. Jiang, P. Stone, and Y. Zhu, “Learning generalizable manipulation policies with object-centric 3D representations,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 3418–3433. [Online]. Available: <https://proceedings.mlr.press/v229/zhu23b.html>
- [8] G. Chen, T. Cui, M. Wang, C. Yang, M. Hu, H. Lu, Y. Mu, Z. Peng, T. Zhou, X. Jiang, Y. Yang, and Y. Yue, “GraphMimic: Graph-to-graphs generative modeling from videos for policy learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 1756–1768. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2025/html/Chen_GraphMimic_Graph-to-Graphs_Generative_Modeling_from_Videos_for_Policy_Learning_CVPR2025_paper.html
- [9] H. Qi, C. Chen, and H. Yang, “Compose by Focus: Scene graph-based atomic skills,” in *IEEE International Conference on Robotics and Automation*, 2026. [Online]. Available: <https://arxiv.org/abs/2509.16053>
- [10] M. Shridhar, L. Manuelli, and D. Fox, “CLIPort: What and where pathways for robotic manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 894–906. [Online]. Available: <https://proceedings.mlr.press/v164/shridhar22a.html>

TABLE I

COMPLETE CoMANI TASK LISTS. SUBTABLES (A) AND (B) RETAIN THE FIXED EVALUATION ORDER AND 0-BASED TASK IDs; ID/OOD DENOTES SEEN/HELD-OUT ATOMIC COMBINATIONS. SUBTABLE (C) LISTS ALL 12 UNSEEN SEQUENCES IN PROVISIONAL ORDER PENDING THE EXECUTABLE EVALUATION LIST. ATOMIC TASKS USE 30 EVALUATION TRIALS EACH, AND SEQUENCES USE 10.

(a) CoMani-Modifier			(b) CoMani-Action		
Task ID	Task instruction	Split	Task ID	Task instruction	Split
0	Put the bowl on the plate.	OOD	0	Put the bowl to the right of the plate.	ID
1	Put the bowl on the stove.	ID	1	Put the bowl to the right of the stove.	ID
2	Put the bowl on the wooden cabinet.	ID	2	Put the cream cheese to the right of the plate.	ID
3	Put the bowl to the right of the plate.	ID	3	Put the cream cheese to the right of the stove.	OOD
4	Put the bowl to the right of the stove.	OOD	4	Rotate the cream cheese right.	OOD
5	Put the bowl to the right of the wooden cabinet.	ID	5	Rotate the stove knob right.	ID
6	Put the cream cheese on the plate.	ID	6	Sweep the bowl to the right of the plate.	OOD
7	Put the cream cheese on the stove.	OOD	7	Sweep the bowl to the right of the stove.	ID
8	Put the cream cheese on the wooden cabinet.	ID	8	Sweep the cream cheese to the right of the plate.	ID
9	Put the cream cheese to the right of the plate.	ID	9	Sweep the cream cheese to the right of the stove.	ID
10	Put the cream cheese to the right of the stove.	ID	Rotation instructions retain the dataset term “right,” which denotes counterclockwise rotation about world +Z when viewed from above.		
11	Put the cream cheese to the right of the wooden cabinet.	OOD			
12	Put the wine bottle on the plate.	ID			
13	Put the wine bottle on the stove.	ID			
14	Put the wine bottle on the wooden cabinet.	OOD			
15	Put the wine bottle to the right of the plate.	OOD			
16	Put the wine bottle to the right of the stove.	ID			
17	Put the wine bottle to the right of the wooden cabinet.	ID			
(c) CoMani-Seq					
Row	Set	Step 1	Step 2	Step 3	Atomic IDs
1	Seq-ID	Bowl on wooden cabinet	Cream cheese right of stove	Wine bottle on plate	2 → 10 → 12
2	Seq-ID	Bowl on wooden cabinet	Wine bottle on plate	Cream cheese right of stove	2 → 12 → 10
3	Seq-ID	Cream cheese right of stove	Bowl on wooden cabinet	Wine bottle on plate	10 → 2 → 12
4	Seq-ID	Cream cheese right of stove	Wine bottle on plate	Bowl on wooden cabinet	10 → 12 → 2
5	Seq-ID	Wine bottle on plate	Bowl on wooden cabinet	Cream cheese right of stove	12 → 2 → 10
6	Seq-ID	Wine bottle on plate	Cream cheese right of stove	Bowl on wooden cabinet	12 → 10 → 2
7	Seq-OOD	Bowl on wooden cabinet	Cream cheese on stove	Wine bottle on plate	2 → 7 → 12
8	Seq-OOD	Bowl on wooden cabinet	Wine bottle on plate	Cream cheese on stove	2 → 12 → 7
9	Seq-OOD	Cream cheese on stove	Bowl on wooden cabinet	Wine bottle on plate	7 → 2 → 12
10	Seq-OOD	Cream cheese on stove	Wine bottle on plate	Bowl on wooden cabinet	7 → 12 → 2
11	Seq-OOD	Wine bottle on plate	Bowl on wooden cabinet	Cream cheese on stove	12 → 2 → 7
12	Seq-OOD	Wine bottle on plate	Cream cheese on stove	Bowl on wooden cabinet	12 → 7 → 2

Execute Step 1, Step 2, and Step 3 in order; every step uses put, and atomic IDs refer to subtable (a). Seq-ID combines 3 ID tasks; Seq-OOD replaces atomic task 10 with OOD task 7.

- [11] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “VIMA: Robot manipulation with multimodal prompts,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 14 975–15 022. [Online]. Available: <https://proceedings.mlr.press/v202/jiang23b.html>
- [12] J. A. Mendez, M. Hussing, M. Gummadi, and E. Eaton, “CompoSuite: A compositional reinforcement learning benchmark,” in *Proceedings of The 1st Conference on Lifelong Learning Agents*, ser. Proceedings of Machine Learning Research, vol. 199. PMLR, 2022, pp. 982–1003. [Online]. Available: <https://proceedings.mlr.press/v199/mendez22a.html>
- [13] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [14] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022. [Online]. Available: <https://arxiv.org/abs/2112.03227>
- [15] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, “C-Pack: Packed resources for general chinese embeddings,” in *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. [Online]. Available: <https://arxiv.org/abs/2309.07597>
- [16] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “SAM 2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.00714>
- [17] S. Jung, A. Zholus, M. Sundermeyer, C. Doersch, R. Goroshin, D. J. Tan, S. Chandar, R. Triebel, and F. Tombari, “TAPNext++: What’s next for tracking any point (TAP)?” *arXiv preprint arXiv:2604.10582*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.10582>
- [18] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” *arXiv preprint arXiv:2210.02747*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.02747>
- [19] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 1820–1864. [Online]. Available: <https://proceedings.mlr.press/v229/mandlekar23a.html>